



How to Compose Micro Frontends at Build Time

Vishal Sharma · [Follow](#)

3 min read · Aug 5, 2022



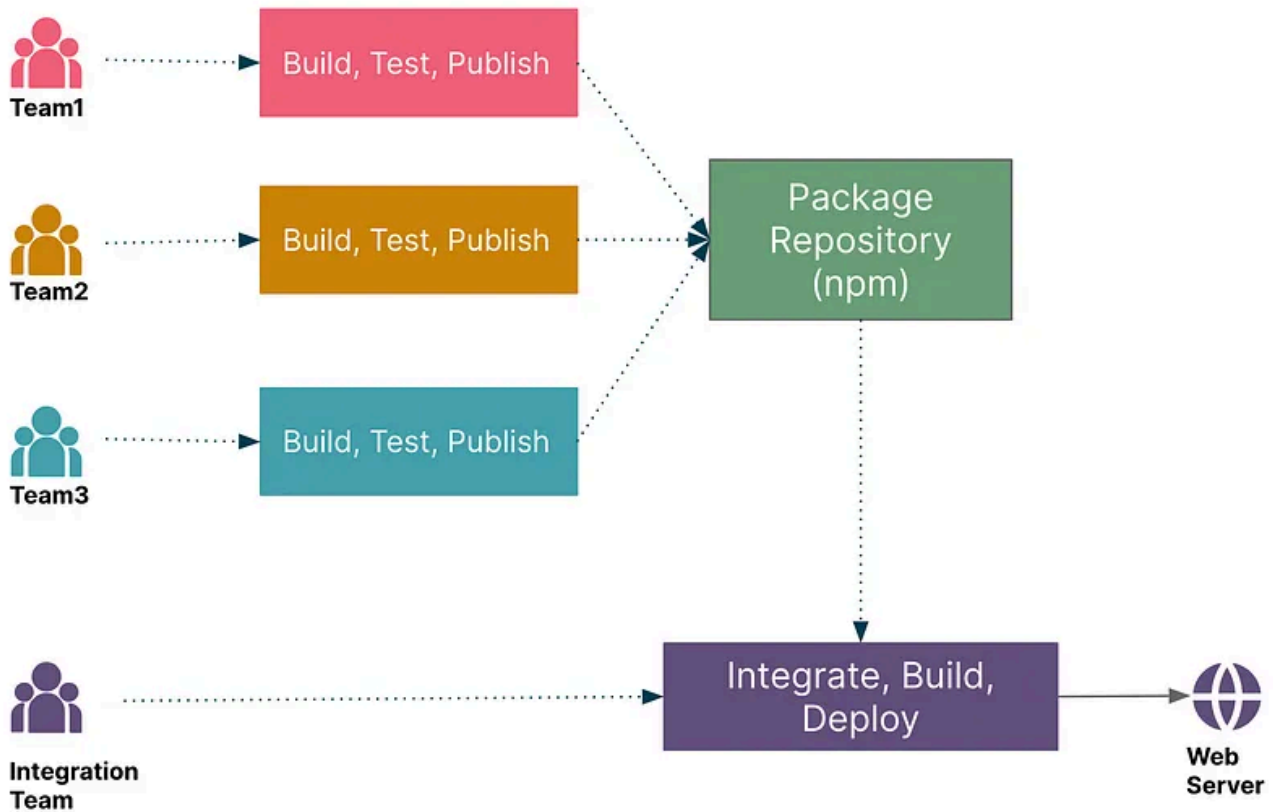
Listen



Share

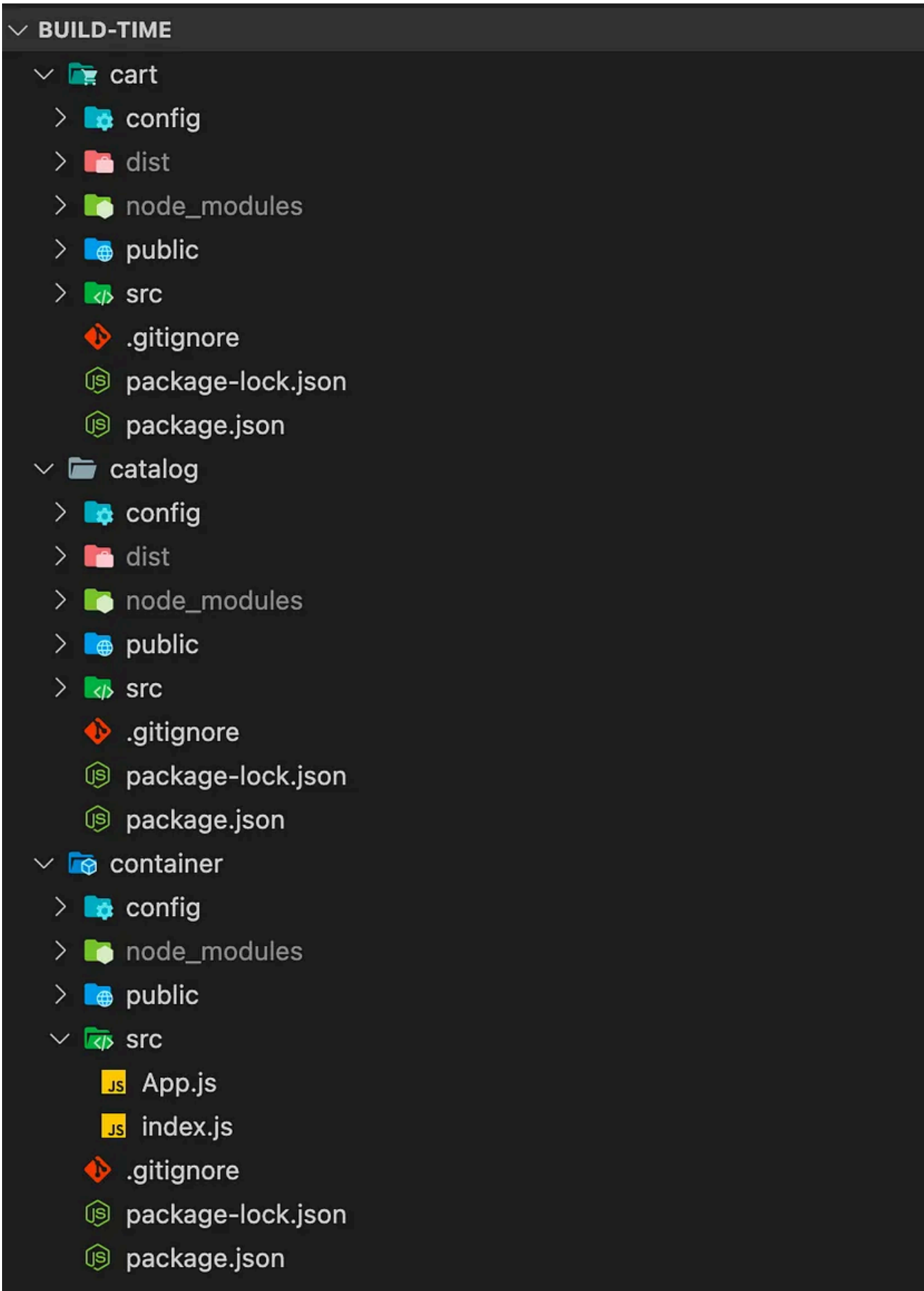
We have been building npm packages and using those inside the frontend application for eternity now. Build time composition of Micro Frontends is nothing but creating micro frontend modules which can run independently but are published as npm packages instead of deployed as a javascript bundle. These modules can then be installed as an npm dependency inside a container application and rendered wherever required.

The orchestrating pattern is called as build time composition of micro frontends because the container with the installed micro frontends becomes the single deployable application. Releasing a change from a micro frontend would involve the following steps:



1. Publish a new version of the micro frontend as an npm package to the npm repository. Each micro frontend should have its own independently running continuous integration pipeline which should publish the new version of the micro frontend in the end.
2. Update the micro frontend package version in the container app.
3. Generate and deploy the container app bundle now containing the latest changes from the micro frontend as well.

Now, let us walk through a simple e-commerce application developed using this pattern.



- The **Catalog** micro frontend shows the list of products available and gives the capability to add a product to the cart

- The **Cart** micro frontend then displays the products added to the cart. For the sake of simplicity, the added products are stored in the browser's storage and read in the cart micro frontend.
- The **Container** is the actual application responsible for installing the Catalog and Cart micro frontends as npm packages and rendering those at appropriate places. The dependencies section of the package.json file of the container looks something like this:

```
"dependencies": {  
  "@material-ui/core": "^4.11.0",  
  "@material-ui/icons": "^4.9.1",  
  "@mfe-patterns/catalog": "1.0.3",  
  "@mfe-patterns/cart": "1.0.1",  
  "react": "^17.0.1",  
  "react-dom": "^17.0.1",  
  "react-router-dom": "^5.2.0"  
},
```

Once installed in the container, the micro frontends can be consumed as:

```
import Catalog from '@mfe-patterns/catalog';  
import Cart from '@mfe-patterns/cart';
```

By looking at the directory structure, we can notice that each micro frontend has its configuration and can be developed and published independently.

The whole code of the above app can be found on [this Github repo](#).

Since we have understood the integration pattern now, let's discuss some pros and cons of it:

Pros

- A very simple and very well-known technique in component-based architecture.
- Easy to get started with. Don't need to worry about the setup cost included in the Run Time composition of micro frontends.
- Micro frontends can be lazily loaded in the container app enhancing the performance of the application.

```

containerWithLazyLoadedMFEs

const CatalogMFE = React.lazy(() => import('@mfe-patterns/catalog'));
const CartMFE = React.lazy(() => import('@mfe-patterns/cart'));

const Catalog = () => (
  <Suspense fallback=<div>Loading...</div>>
    <CatalogMFE />
  </Suspense>
);

const Cart = () => (
  <Suspense fallback=<div>Loading...</div>>
    <CartMFE />
  </Suspense>
);

```

- This is the only available option to build micro frontend architecture in cross-platform solutions like React Native.
- A design library exposed as an npm package can lead to design consistency in the micro frontend architecture.

Cons

- Even though the micro frontends can run independently, once changed and published, the version has to get updated and deployed in the container. This can lead to a lot of communication overhead.
- The container should only be responsible for rendering the micro frontends. If the data end up flowing from one micro frontend to another via container application i.e. if a lot of coupling is introduced, then the application will just be another **Distributed Monolith**.

I would end this article by mentioning that this composition pattern is very convenient in the case of mobile micro frontends or when we need to build reusable modules. Since the pattern can lead to coupling issues between micro frontends and container, this should only be used when Run time integration of micro frontends is not possible.

To understand which micro frontend communication technique can suit this pattern, check out the article on [5 different techniques for cross micro frontend communication](#).

Micro Frontends

Build Time

Npm Module

Lazy Loading

React



Follow

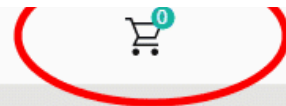


Written by Vishal Sharma

129 Followers

Front End Engineer at /thoughtworks.

More from Vishal Sharma



Tennis Shoes

129.95 **99.95**

ADD TO CART



Running Shoes

129.95 **99.95**

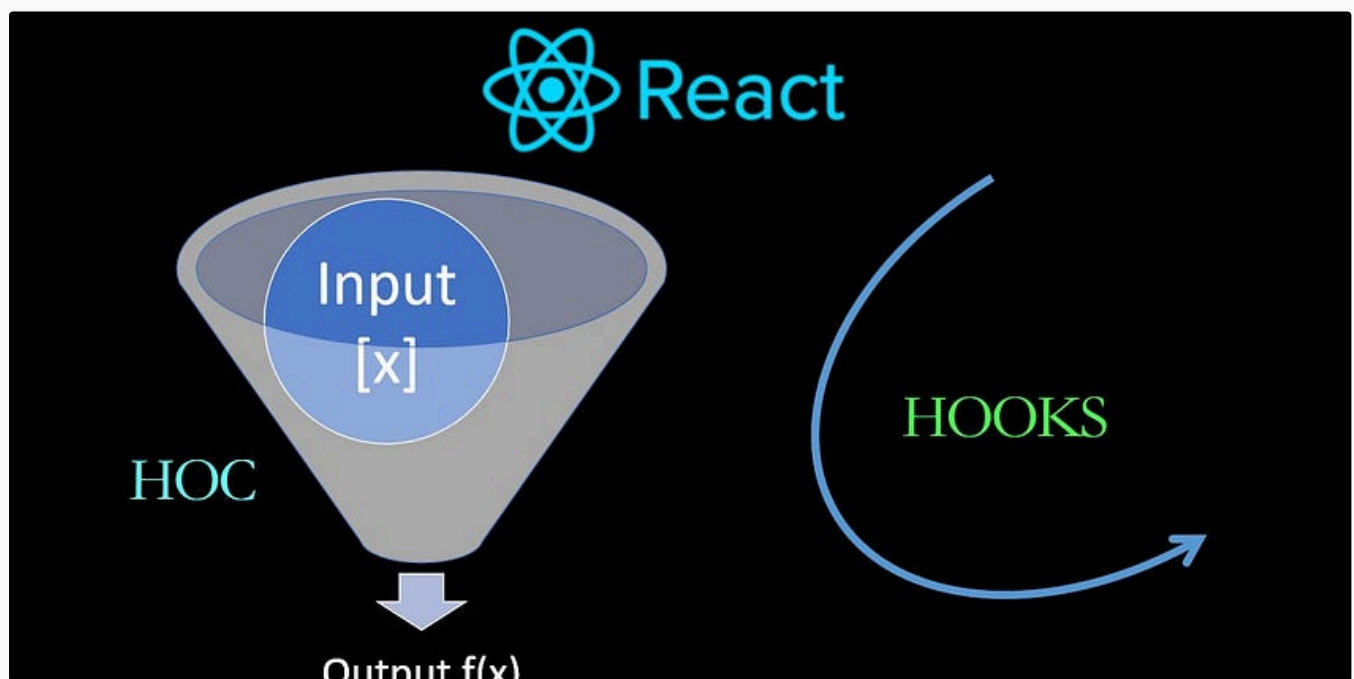
ADD TO CART

 Vishal Sharma

5 Different Techniques for Cross Micro Frontend Communication

The title itself signifies what we are going to discuss in this article. But before we even start, let's try to understand why cross-micro...

Jun 12, 2022  247  4



 Vishal Sharma

Following DRY principle for Api Calls in React

In most of the React projects, we have components where we need to fetch the data from an api on mount. React provides us lifecycle...



```
package.json

{
  "name": "ts-eslint-jest-setup",
  "version": "1.0.0",
  "main": "index.js",
  "license": "MIT"
}
```

 Vishal Sharma

Setting up Typescript + EsLint + Jest Project

A lot of node projects have a setup which usually include Typescript for static type checking, esLint for static code analysis and Jest as...



 Vishal Sharma

SSR and Web Components— A Contentious Couple

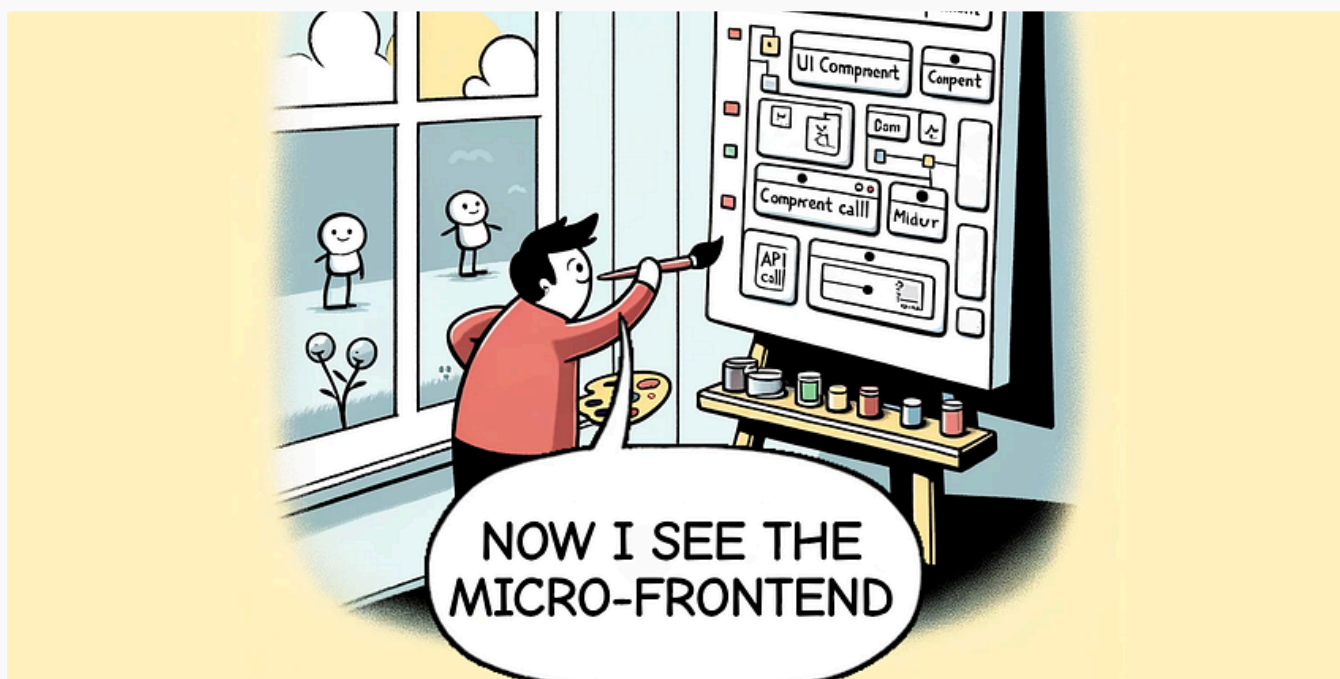
Usually, we talk about the technologies that work best together. This article is the exact opposite where I would like to discuss the...

Jul 31 🖱 9



See all from Vishal Sharma

Recommended from Medium



David Rodenas PhD

Overcoming the Implementation Challenges of Micro-Frontends

A comprehensive guide of the micro-frontend architecture and implementation challenges.



Mar 30



137



3





 Asian Digital Hub

React Component Reusability in Micro-Frontends

Are you tired of reinventing the wheel every time you build a new micro-frontend? Do you find yourself copying and pasting React components...

🌟 Sep 28



Lists



Stories to Help You Grow as a Software Developer

19 stories · 1419 saves



General Coding Knowledge

20 stories · 1647 saves



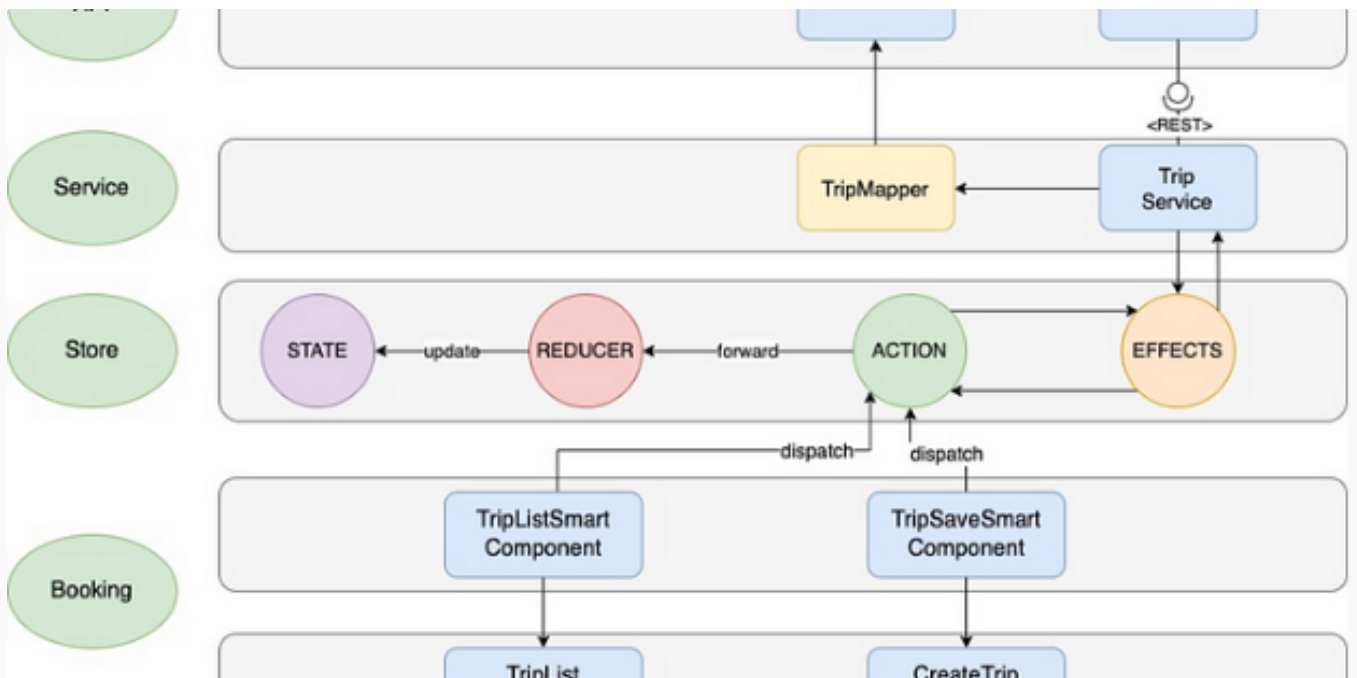
Medium's Huge List of Publications Accepting Submissions

334 stories · 3692 saves



Natural Language Processing

1754 stories · 1351 saves

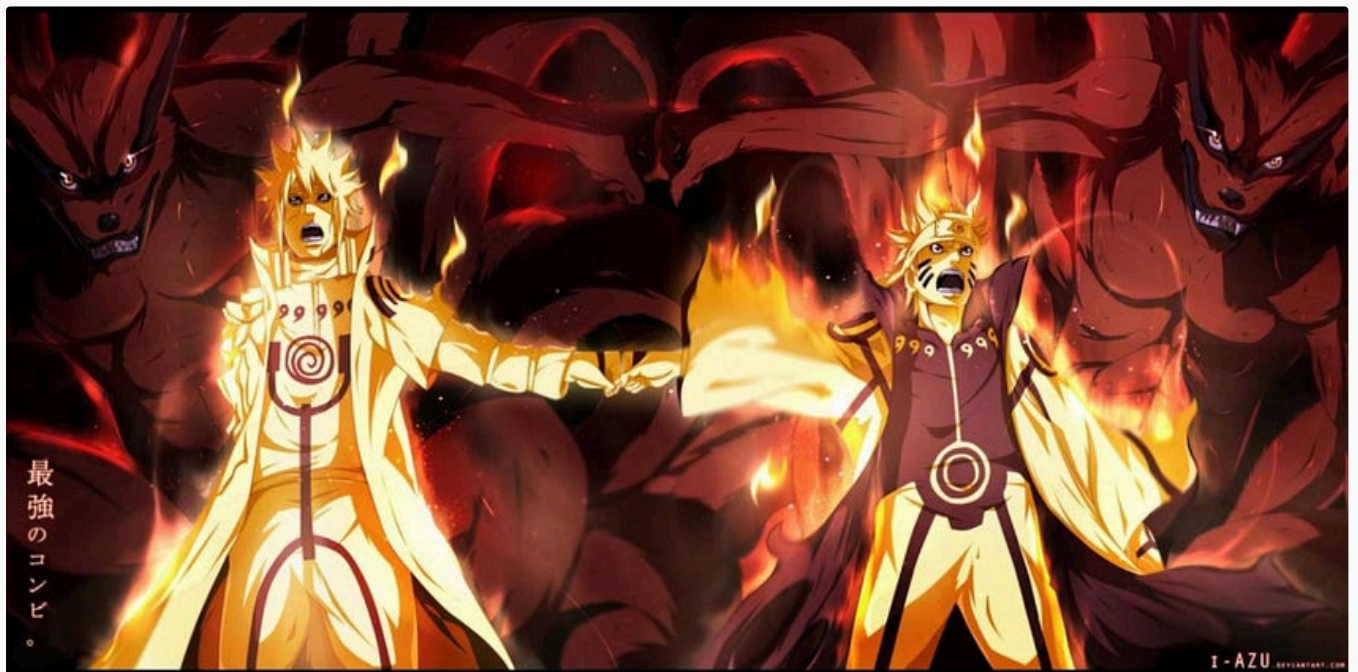


J.D Christie

Building a Clean and Scalable Frontend Architecture

Digital and technological development continues unstoppable, so, if we talk about the design of web applications, we cannot lose sight of...

🌟 Jan 13 🖱️ 1.1K 💬 7



Guhaprasanth Nandagopal

React vs. Next.js: Microfrontend Matchup—Making Them Play Nice with Vite

Microfrontend (MFE) architecture allows you to build complex applications by composing multiple smaller frontend applications, each...

✦ Jun 14 🖱 26



 Tari Ibaba in Coding Beauty

5 amazing new JavaScript features in ES15 (2024)

5 juicy ES15 features with new functionality for cleaner and shorter JavaScript code in 2024.

✦ Jun 2 🖱 3K 💬 20



 Arunangshu Das

Handling Routing in a Microfrontend Architecture

As modern web applications grow in complexity, the monolithic approach can become a bottleneck for development speed, scalability, and...

Jun 2  6



See more recommendations